

渲染与精灵子系统 · 答辩速查卡（单页速查）

一、项目总结

Q：解决什么问题？
 局域网多机协同鱼群动画；鱼从一块屏幕游到另一台机器继续游动 + 精灵按需全网同步

Q：架构分几层？
 网络层 network.py/message.py；资源层 sprite/background/audio_manager；实体层 fish_entity；渲染层 renderer/ui；主循环 main.py

Q：为什么用 pygame？
 教学场景要交可读源码，pygame 纯 Python+几百行能跑通，跨平台无依赖，答辩现场可读代码

二、Fish 实体类

Q：fish_id 编码 (host_id<<8 | counter)
 高 8 位放主机号→跨机天然隔离；低 8 位本机自增→单机 ≲256 条足够；两台 host_id 都从 0 起鱼 ID 不会撞

Q：in_bridge vs transfer_cooldown？
 in_bridge=几何状态（穿越中不反弹）；transfer_cooldown=协议状态（2s 内禁发 TRANSFER）。一个管物理一个管网络协议，必须独立

Q：bounce 反射公式为什么 $\pi \cdot d / -d$ ？
 左右墙：x 反向、y 保持→ $\pi \cdot \text{direction}$ （关于 x 轴对称）；上下墙：y 反向、x 保持→ direction （关于 y 轴对称）

Q：为什么反弹加 [-0.3, 0.3] 随机扰动？
 避免多鱼‘对齐泳’周期性同步；0.3 弧度 $\approx \pm 17^\circ$ 打破周期但视觉仍自然

Q：碰撞半径为什么 $\max(w,h)/2$ ？
 保守估计（最长边外接圆），保证不穿透；鱼身多数横向长，正方形外接最稳

Q：动画帧周期 0.15s？
 <0.1s 抽搐；>0.2s 僵硬；0.15s $\approx 6\text{--}7\text{Hz}$ 鱼鳍摆动，与真实鱼类节奏接近

Q：size*0.7 的 0.7 是什么？
 精灵归一化到 256² 透明画布的视觉修正系数；给鱼鳍/尾鳍留透明 padding；改 1.0 鱼会‘太满’

Q：为什么 HSV 采样颜色？
 HSV 限制 $s \in [0.6, 0.9]$ 、 $v \in [0.7, 1.0]$ 排除灰暗色；RGB 全随机约 1/4 落在‘脏’区间

三、SpriteManager

Q：为什么每鱼种一个子目录？
 平铺方式要带类型前缀易混；子目录天然隔离，类型名=目录名；跨平台都支持含空格目录名

Q：为什么预生成 flipped_frames 缓存？
 transform.flip 每帧 N 次=N 次 Surface 分配；预生成一次→运行时 O(1) 索引；CPU 25%→8%

Q：导入精灵为什么归一化 256×256 透明画布？
 renderer 避用统一公式 $\text{size} \cdot 0.7$ 缩放+计算碰撞；居中等比缩放，短边留透明 padding 避免边缘裁切

Q：等比缩放用 $256/\max(w,h)$ 而非 min？
 长边归一保证所有精灵不超出 256² 边界；短边按比例缩放，居中后留 padding

Q：三层 fallback 设计理由？
 网络同步：未知 fish_type 不能让游戏崩；type 存在→正常；type 缺失→用第一 type；无素材→粉红方块；‘降级而非崩溃’是长生命周期程序标配

Q：_migrate_if_needed 一次性迁移怎么幂等？
 判定条件：老目录存在 AND 新位置不存在；shutil.copy2 复制而非移动，老文件保留可回滚

四、SpriteSyncManager

Q：为什么用位掩码 (1<<ci) 而非 len==total？
 容忍乱序和重复到达（OR 幂等）；len==total 在‘丢 1 收 2’时误判；位掩码是网络协议经典做法（XMODEM / HDLC）

Q：MAX_PENDING=32 为什么是 32？
 每 entry ~200-500B×32 $\approx$ 16KB 可忽略；32 个并发足够 32 对端同时拉精灵的极端场景；超限说明对端掉线，老 entry 优先淘汰

Q：chunk 丢失怎么办？
 当前不主动重传，不完整时该帧不落地；对端 SPRITE_PING 会重新触发对方广播，本地可再发 SPRITE_REQ；‘被动重传’简单可靠

Q：为什么文件名 Fish-[frame_index+1].png（0→1）？
 内部协议 0 基便于 mask 位运算；文件名 1 基符合用户直觉；两边约定不冲突

五、BackgroundManager

Q：为什么做三种模式？
 教室投影→渐变（干净）；PC 演示→图片（氛围）；期末展示→视频（吸睛）；一套接口让 UI 切换零成本

Q：import cv2 为什么要 lazy？
 cv2 包体 >50MB、启动慢数百毫秒；绝大多数用户只用 gradient/image；lazy import 让‘不启用视频=零代价’

Q：cv2 帧怎么转 pygame Surface？
 cv2 默认 BGR，先 cvtColor(COLOR_BGR2RGB)；surfarray.make_surface 把 numpy 转 Surface；swa paxes(0,1) 是 numpy (H,W,C)→pygame (W,H)

Q：视频播完怎么办？
 读不到下一帧时 CAP_PROP_POS_FRAMES=0 重置循环；循环也失败→_release_video() 退到 gradient；主动选的视频不主动退，损坏/解码失败才退

Q：on_resize 为什么同时失效渐变缓存又重缩放 image？
 渐变按 (w,h) 像素逐行算必须重画；image 预缩放绑定屏幕尺寸，全屏切换旧图有黑边；两者同步刷新

六、Renderer 多级缓存

Q：4 级缓存分别缓存什么？
 _hud_font=SysFont 实例（永不失效）；_hud_bg=160×92 半透明黑底 strip（永不失效）；_hud_text[key]=文字 Surface（值变化重 render）；fish_scaled_cache=smoothscale 后的鱼图

Q：Font 缓存为什么必要？
 每帧创建 SysFont 在某些 SDL 平台会泄漏 C 层 TTFFont；复用一份既省 CPU 又省内存

Q：FPS 节流到 2Hz 不会‘跳’吗？
 会跳但人眼察觉不到；60Hz 刷新反而‘读不过来’；用视觉冗余换 CPU 的标准 trade-off

Q：_scaled_cache key 为什么不含 size？
 size 变化时整条鱼视觉完全变，旧缓存对不上→失效正确；size 不变时 100% 命中（这是常态）；不含 size 反而让 0.6-1.4 区间稳定命中

七、性能与改进 + 应试技巧

Q：怎么量化优化效果？
 transform.smoothscale：~1200 次/秒→~0 次/秒；800×600+20 条鱼：CPU 25%→8%；60fps 更稳

Q：进一步优化方向？
 短期：SpriteSync 加主动重传；中期：lz4 压缩 PNG，chunk 数降 60%；长期：消息换 protobuf，带宽再降 30-50%

Q：当前已知不足？
 UDP 无重传，精灵同步失败只能等下次 SPRITE_PING；transfer 期间鱼可能因反弹飞出屏幕；>50 条鱼时 alpha 合成 CPU 飙升

Q：现场读陌生代码的方法论？
 ①看函数签名（输入输出契约）；②看 docstring（作者意图）；③看 if/else 分支（条件差异）；④代入最小例子手算一遍

Q：被质疑‘只是调 API’怎么回击？
 举一个细节反向证明深度；例：位掩码完整性→1980s 网络协议栈经典做法→expected_mask=(1<<total)-1 制造低 total 位全 1 掩码→total>0 才有意义

Q：老师问‘还有什么不足’怎么答？
 分 3 层：已知 bug/性能瓶颈/可靠性；答辩时坦白局限反而显得工程成熟；例：transfer 期间缺 cooldown 检查、UDP 无 checksum

数字	含义	索引 关键实现	文件位置
0.15s	动画帧切换周期	256 ID 编码	fish_entity.py:69-70
0.7	渲染视觉修正系数	460 Bridge 状态	main.py:539-575
70	碰撞 fallback 系数	32 预翻转缓存	sprite_manager.py:144-146
8	fish_id 低位 bit 数	0.3 位掩码判定	sprite_sync.py:88-93
2.0s	transfer_cooldown	2Hz 4 级缓存	renderer.py:30-40, 82-122
		cv2 帧转换	background_manager.py:224-231
		Lazy import	background_manager.py:107